LAPORAN PRAKTIKUM TEKNOLOGI IOT II

TUGAS MODUL 8

Penyimpanan Data Sensor ke Time-Series Database (InfluxDB)

Disusun Oleh:

Nama : Naela Deswita

NPM : 25782106

Kelas : 3D-TRI

Dosen Pengampu:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

Andi Saptono, S.Kom

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

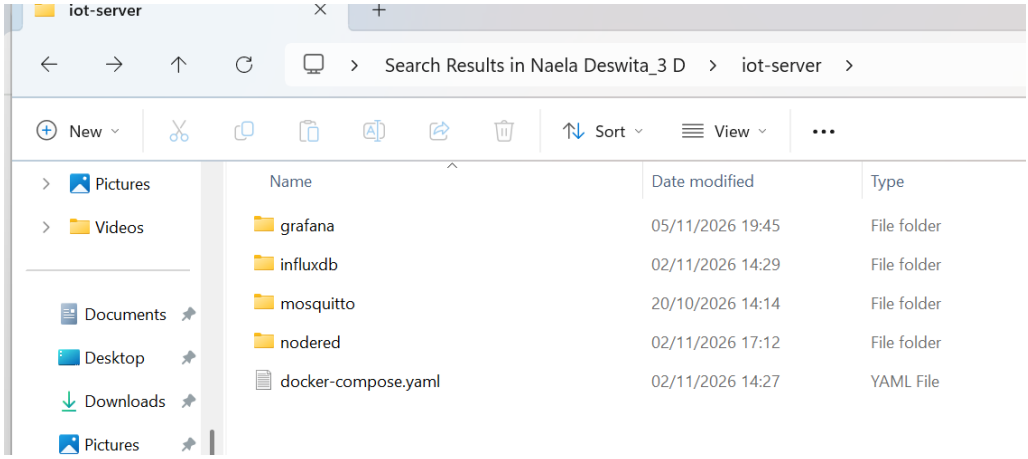
2026/2027

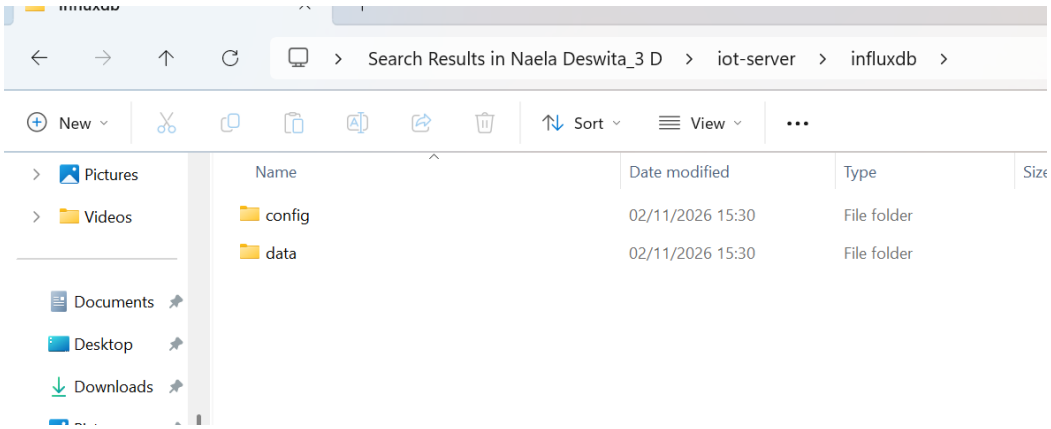
8.5 Pelaksanaan Praktikum Modul 8

8.5.1 Praktikum 1: Peningkatan Ekosistem IoT Server dengan InfluxDB v2 dan Grafana pada Docker Compose

1. Menyiapkan Struktur Folder Proyek

Saya menyiapkan folder proyek iot-server lanjutan dari modul 7 sebagai tempat menyimpan seluruh kebutuhan konfigurasi server IoT. Di dalam folder tersebut terdapat beberapa direktori, yaitu grafana, influxdb, mosquitto, dan nodered, serta file docker-compose.yaml yang digunakan untuk mengatur layanan Docker.





2. Menyiapkan Konfigurasi Docker Compose

Selanjutnya, saya menggunakan file docker-compose.yaml untuk mengatur layanan yang akan dijalankan dalam lingkungan Docker. Konfigurasi ini mencakup InfluxDB sebagai database penyimpanan data sensor, Grafana untuk visualisasi, Mosquitto sebagai MQTT Broker, dan Node-RED untuk mengolah aliran data. saya menambahkan Grafana dan influxdb.

```
  influxdb:
    image: influxdb:2
    container_name: influxdb2
    restart: unless-stopped
    ports:
      - "8086:8086"  # Port HTTP API & Web UI InfluxDB v2
    environment:
      # Inisialisasi otomatis akun administrator dan bucket awal
      DOCKER_INFLUXDB_INIT_MODE: setup
      DOCKER_INFLUXDB_INIT_USERNAME: admin
      DOCKER_INFLUXDB_INIT_PASSWORD: adminpassword
      DOCKER_INFLUXDB_INIT_ORG: my-org
      DOCKER_INFLUXDB_INIT_BUCKET: my-bucket
      DOCKER_INFLUXDB_INIT_RETENTION: 30d
    volumes:
      - ./influxdb/data:/var/lib/influxdb2
      - ./influxdb/config:/etc/influxdb2

  # ============================================================
  # 4. Layanan Dasbor Visualisasi Industri: Grafana
  # ============================================================
  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    restart: unless-stopped
    ports:
      - "3000:3000"  # Port Antarmuka Dasbor Web Grafana
    environment:
      GF_SECURITY_ADMIN_USER: admin
      GF_SECURITY_ADMIN_PASSWORD: adminpassword
      GF_USERS_ALLOW_SIGN_UP: "false"
    volumes:
      - ./grafana:/var/lib/grafana
```

3. Menjalankan Seluruh Container

saya menjalankan perintah docker compose up -d melalui terminal untuk mengaktifkan seluruh layanan yang sudah dikonfigurasi. Setelah proses selesai, saya memeriksa status container menggunakan docker compose ps dan terlihat bahwa layanan Node-RED, InfluxDB, Grafana, serta MQTT Broker berhasil berjalan.

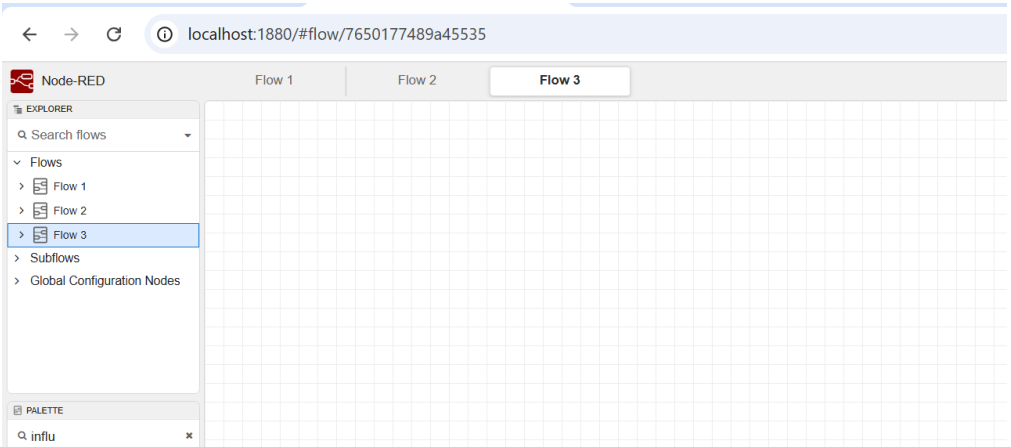
```
D:\Naela Deswita_3 D\iot-server>docker compose up -d
[+] up 4/4
 ✔Container node-red    Running                                                    0.0s
 ✔Container influxdb2   Running                                                    0.0s
 ✔Container grafana     Running                                                    0.0s
 ✔Container mqtt-broker Running                                                    0.0s

D:\Naela Deswita_3 D\iot-server>docker compose ps
NAME          IMAGE                     COMMAND                  SERVICE    CREATED       STATUS                   PORTS
grafana       grafana/grafana:latest    "/run.sh"                grafana    3 days ago    Up 15 minutes            0.0.0.0:3000->30
00/tcp, [::]:3000->3000/tcp
influxdb2     influxdb:2                "/entrypoint.sh infl…"   influxdb   3 days ago    Up 15 minutes            0.0.0.0:8086->80
86/tcp, [::]:8086->8086/tcp
mqtt-broker   eclipse-mosquitto:latest  "/docker-entrypoint.…"   mqtt       2 weeks ago   Up 15 minutes            0.0.0.0:1883->18
83/tcp, [::]:1883->1883/tcp, 0.0.0.0:9001->9001/tcp, [::]:9001->9001/tcp
node-red      nodered/node-red:latest   "./entrypoint.sh"        nodered    2 weeks ago   Up 15 minutes (healthy)  0.0.0.0:1880->18
80/tcp, [::]:1880->1880/tcp

D:\Naela Deswita_3 D\iot-server>
```

4. Memastikan Layanan Berhasil Diakses

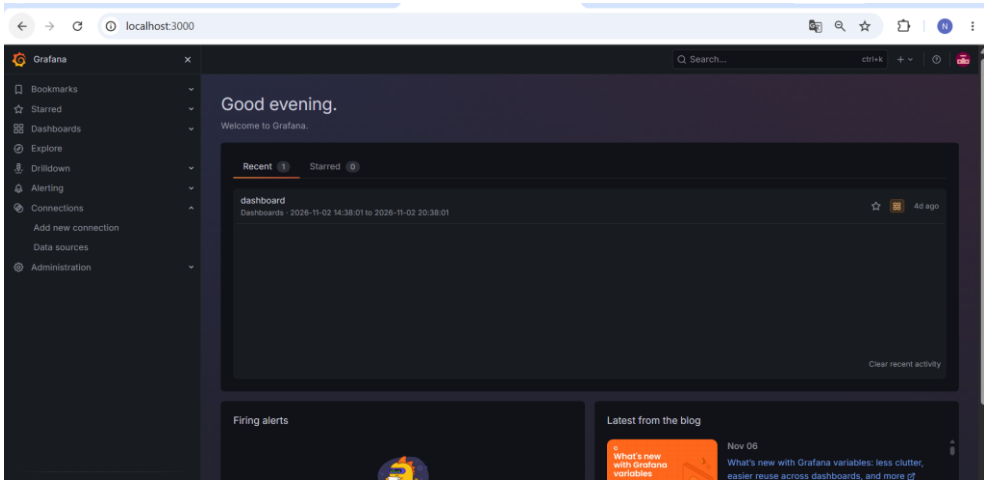
Setelah seluruh container aktif, saya mencoba mengakses layanan melalui browser untuk memastikan server dapat digunakan. Node-RED dapat dibuka melalui localhost port 1880, sedangkan InfluxDB dan Grafana juga dapat diakses melalui antarmuka web masing-masing.



8.5.2 Praktikum 2: Inisialisasi dan Manajemen Kredensial & API Token InfluxDB v2

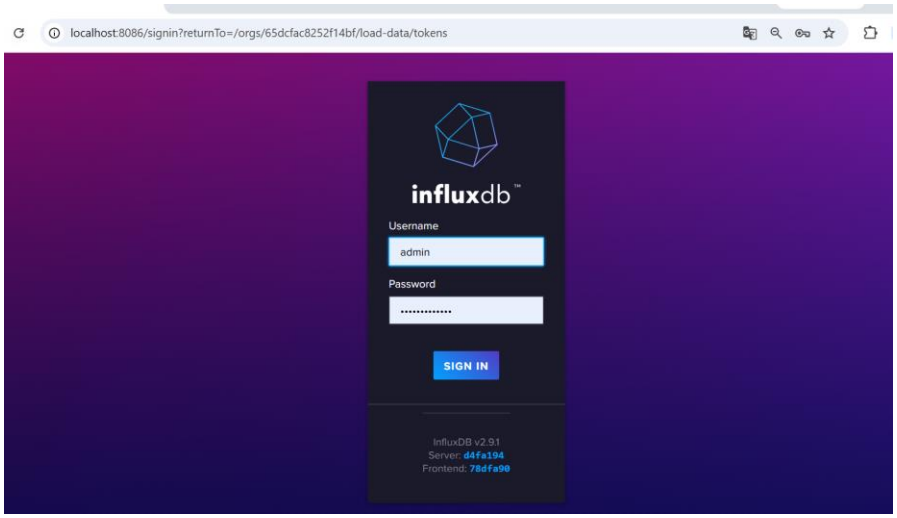
1. Membuka Dashboard Grafana

saya membuka Grafana melalui browser menggunakan alamat localhost:3000. Dashboard Grafana digunakan sebagai media visualisasi untuk menampilkan data yang nantinya diperoleh dari database InfluxDB.



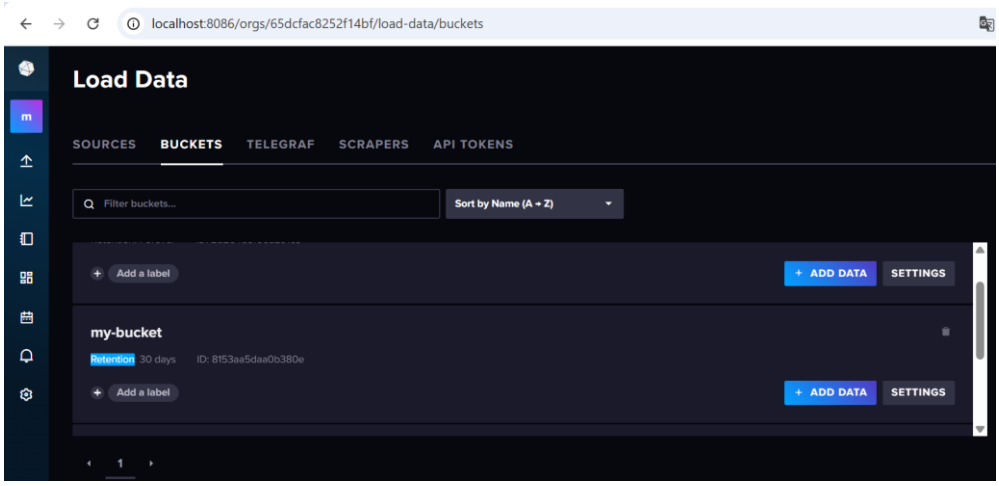
2. Membuka InfluxDB dan Melakukan Login

Selanjutnya, saya membuka InfluxDB melalui browser menggunakan alamat localhost:8086. Saya melakukan login menggunakan username dan password yang telah dikonfigurasi sebelumnya agar dapat mengakses pengaturan database.



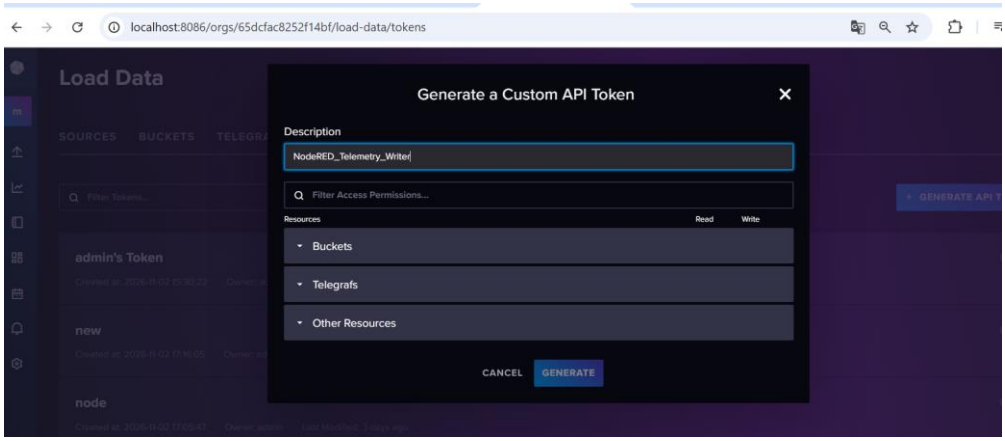
3. Memeriksa Bucket pada InfluxDB

Setelah berhasil masuk, saya membuka menu Load Data kemudian memilih bagian Buckets. Pada halaman ini terlihat bucket my-bucket yang digunakan sebagai tempat penyimpanan data telemetri dari sensor.



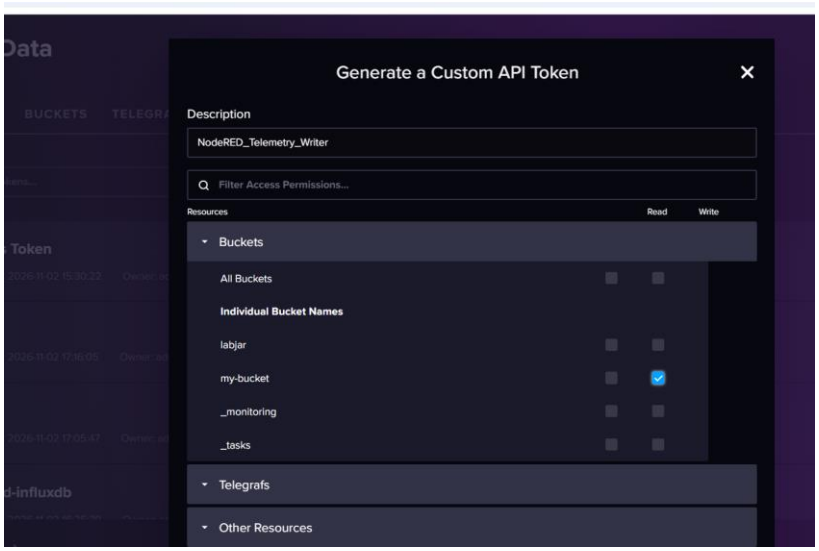
4. Membuat Custom API Token

Berikutnya, saya membuat Custom API Token melalui menu API Tokens pada InfluxDB. Token ini digunakan sebagai autentikasi agar Node-RED dapat mengirimkan dan menyimpan data ke dalam InfluxDB tanpa harus menggunakan kredensial login secara langsung.



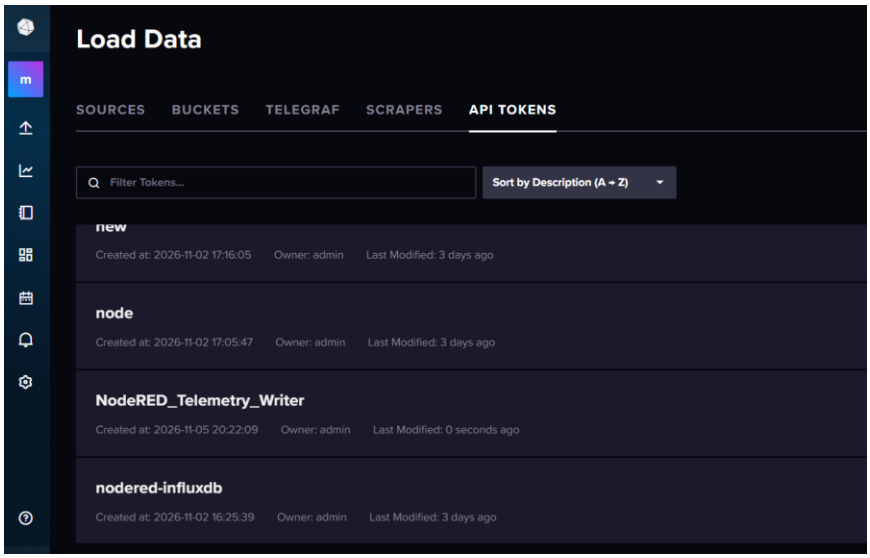
5. Mengatur Hak Akses API Token

Pada tahap ini, saya memberikan izin akses Write pada bucket my-bucket untuk token yang dibuat. Pengaturan tersebut bertujuan agar token hanya memiliki hak untuk menulis data ke bucket yang telah ditentukan.



6. Memastikan API Token Berhasil Dibuat

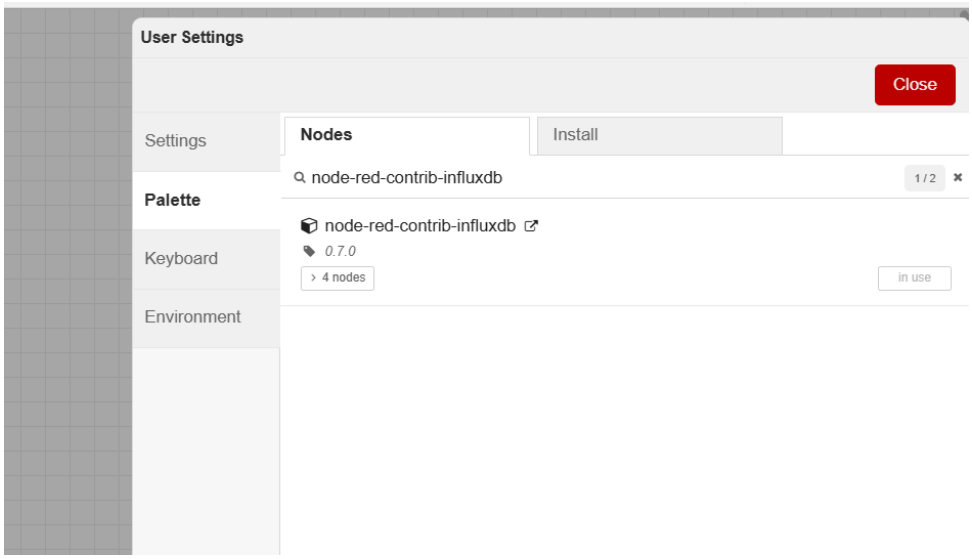
Setelah proses pembuatan selesai, token yang telah dibuat muncul pada daftar API Tokens di InfluxDB. Hal ini menunjukkan bahwa token berhasil dibuat dan dapat digunakan pada konfigurasi Node-RED untuk proses penyimpanan data sensor.



8.5.3 Praktikum 3: Instalasi Node InfluxDB dan Perancangan Pipeline Data pada Node-RED

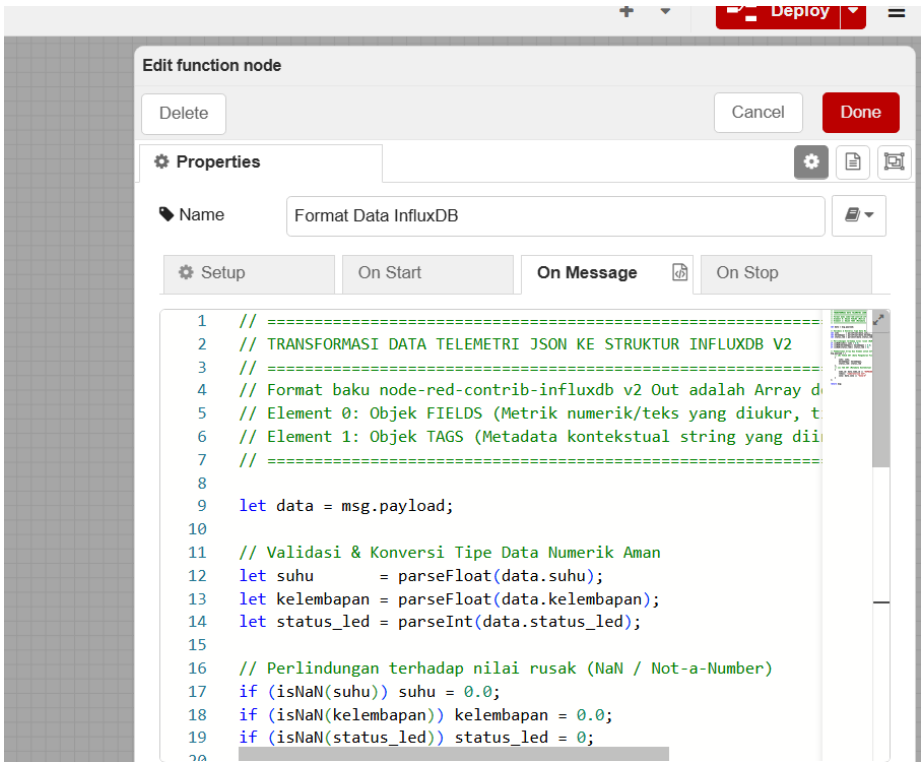
1. Menginstal Node InfluxDB pada Node-RED

saya membuka menu Manage Palette pada Node-RED untuk mencari dan memasang package node-red-contrib-influxdb. Package ini diperlukan agar Node-RED memiliki node khusus yang dapat digunakan untuk mengirimkan data ke database InfluxDB.



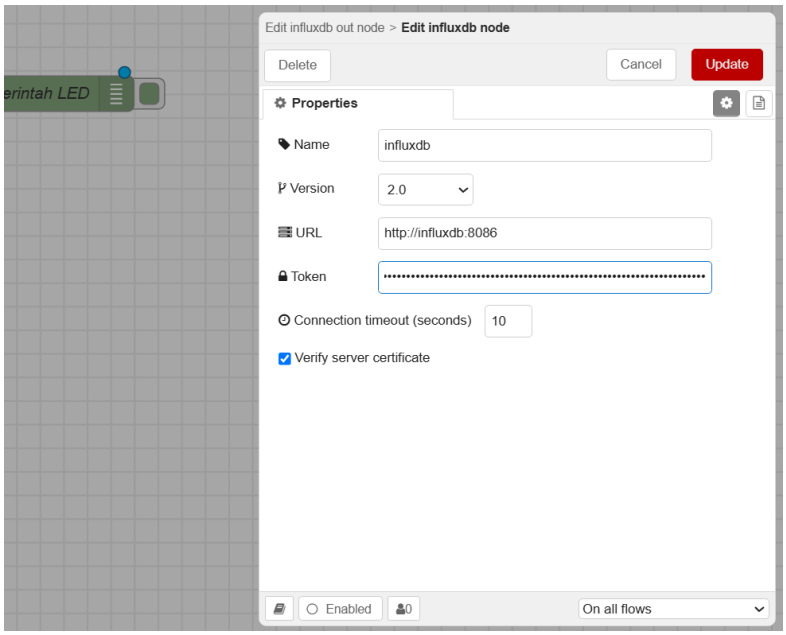
2. Membuat Function untuk Transformasi Data

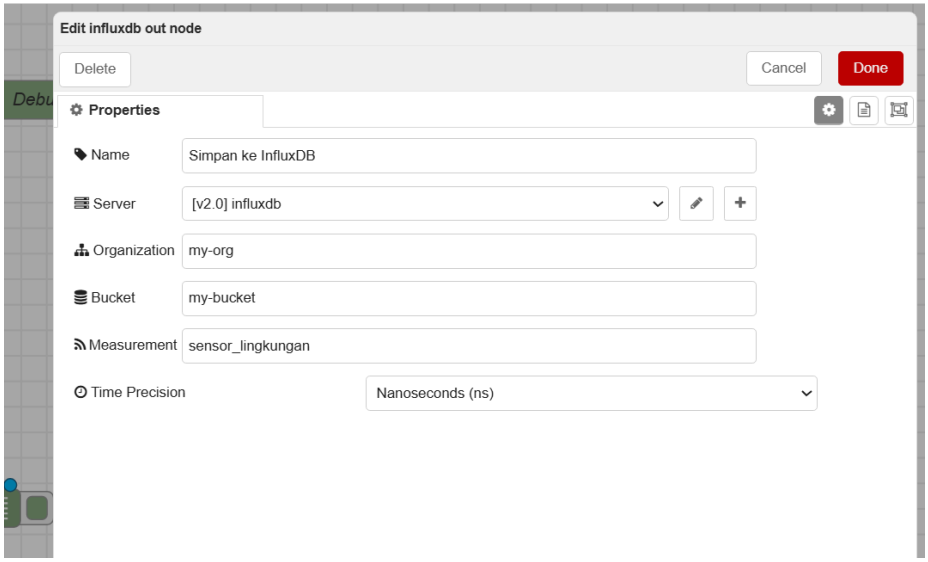
Selanjutnya, saya membuat Function Node dengan nama Format Data InfluxDB untuk mengolah data yang diterima dari MQTT. Function ini digunakan untuk mengambil nilai suhu dan kelembapan, mengubahnya menjadi tipe data numerik, serta menyusun data ke dalam format yang sesuai dengan InfluxDB.



3. Mengatur Konfigurasi Koneksi InfluxDB

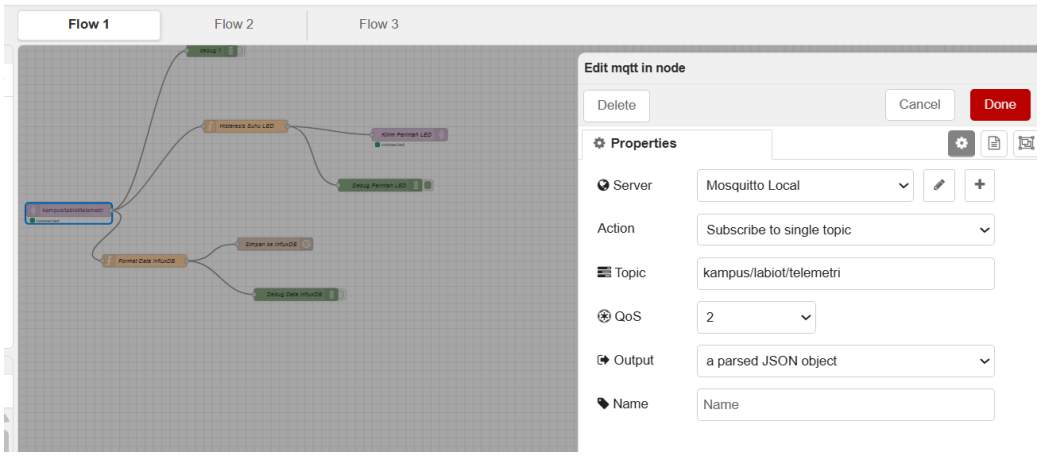
Pada tahap ini, saya mengatur konfigurasi InfluxDB Out Node dengan menggunakan versi InfluxDB 2.0. Saya memasukkan alamat server http://influxdb:8086, Organization my-org, bucket my-bucket, serta API Token yang sebelumnya telah dibuat agar Node-RED dapat terhubung ke database.





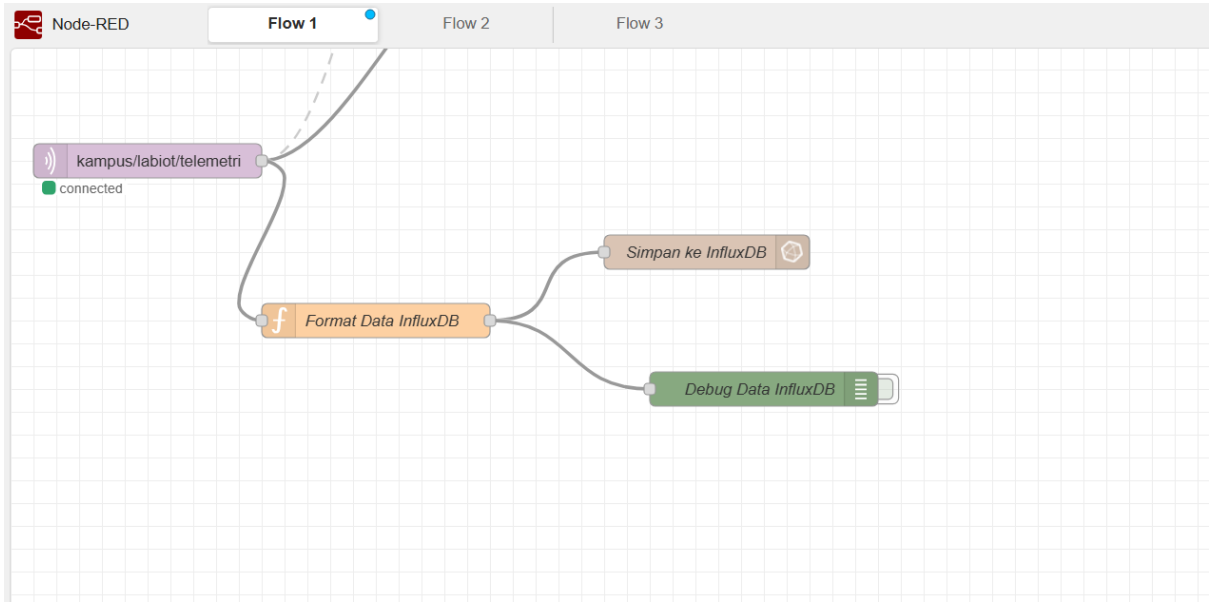
4. Mengatur MQTT Input

Selanjutnya, saya mengatur MQTT Input Node untuk menerima data telemetri yang dikirimkan oleh perangkat NodeMCU melalui MQTT Broker. Topic yang digunakan adalah kampus/labiot/telemetri dengan QoS 2 dan output berupa objek JSON agar data dapat diproses oleh Function Node.



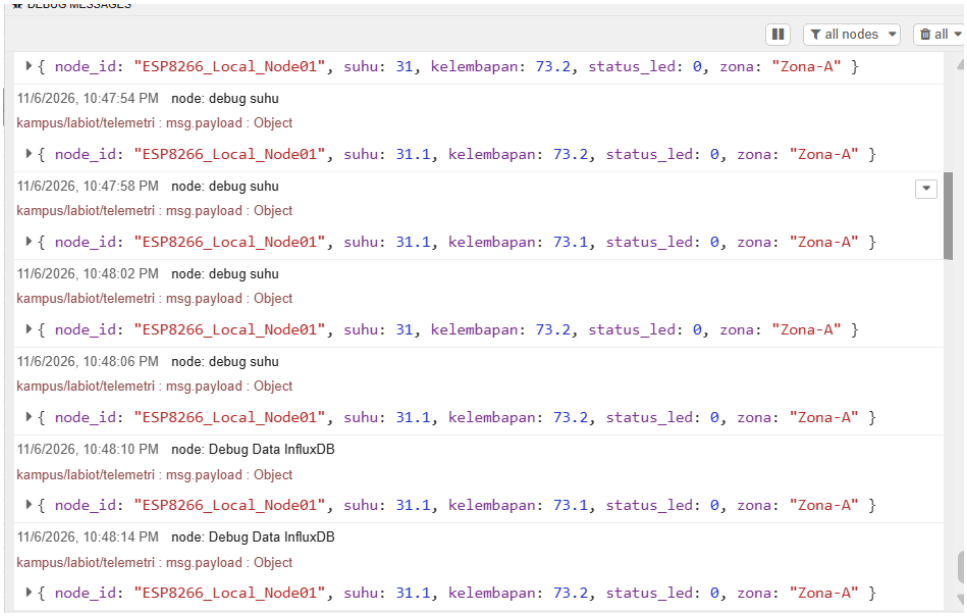
5. Menyusun Alur Pipeline Data

Setelah setiap node dikonfigurasi, saya menghubungkan MQTT Input ke Function Node Format Data InfluxDB. Keluaran Function kemudian dihubungkan ke InfluxDB Out untuk menyimpan data, serta Debug Node untuk memeriksa hasil transformasi sebelum data dikirimkan ke database.

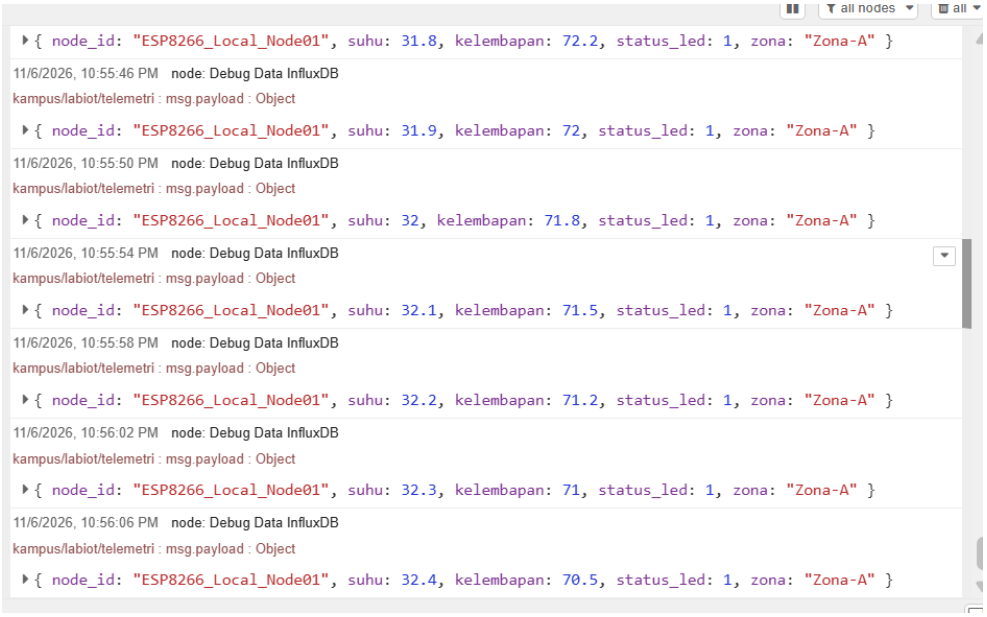


6. Melakukan Deploy dan Memeriksa Output Debug

Setelah seluruh node terhubung, saya menekan tombol Deploy untuk menerapkan konfigurasi yang telah dibuat. Pada panel Debug terlihat data sensor berhasil diterima, termasuk nilai suhu dan kelembapan, serta status koneksi MQTT menunjukkan kondisi connected.



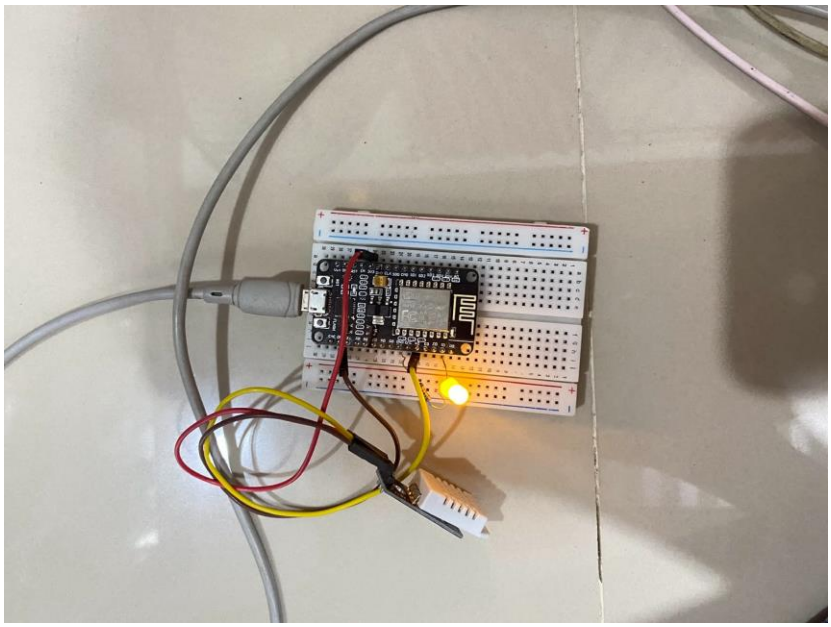
Berdasarkan hasil Debug, terlihat bahwa data suhu dan kelembapan berhasil diterima secara berkala melalui komunikasi MQTT. Hasil ini menunjukkan bahwa proses penerimaan data dari perangkat dan pemantauan melalui Node-RED telah berjalan.

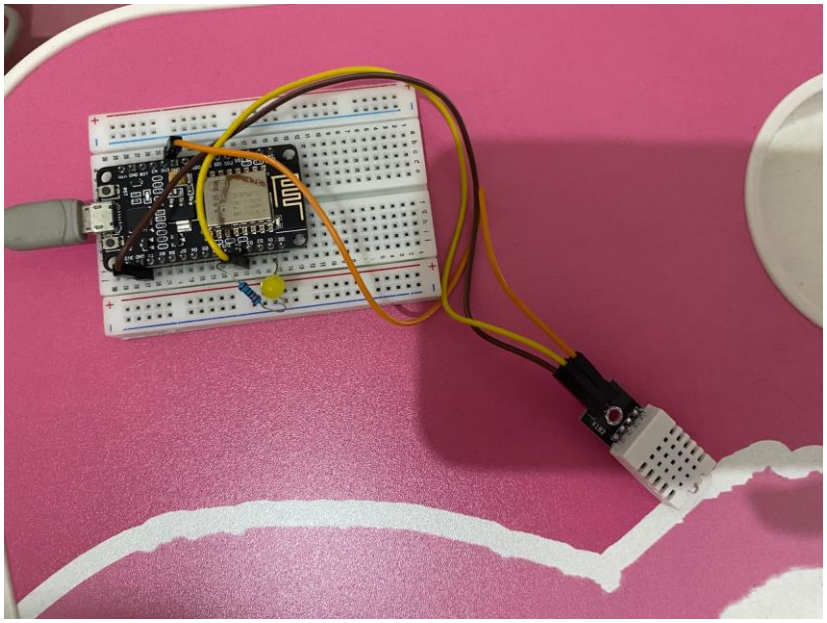


8.5.4 Praktikum 4: Integrasi End-to-End NodeMCU ESP8266 dan Verifikasi Telemetri Time-Series

1. Menyiapkan Rangkaian Perangkat IoT

Pada tahap ini, saya menyiapkan rangkaian perangkat menggunakan NodeMCU ESP8266 yang dihubungkan dengan sensor DHT dan LED. Rangkaian tersebut digunakan untuk membaca kondisi suhu dan kelembapan lingkungan sekaligus menguji pengendalian LED berdasarkan nilai suhu yang diperoleh.





2. Membuat dan Mengunggah Program Arduino

Selanjutnya, saya menyiapkan program pada Arduino IDE untuk mengatur koneksi Wi-Fi, MQTT Broker, pembacaan sensor DHT, serta pengendalian LED. Program tersebut juga mengatur pengiriman data telemetri secara berkala agar informasi dari sensor dapat diterima oleh sistem.

kode8.5.4.ino

```
 * ========================================================================
 */

#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>

// 1. Kredensial Jaringan Wi-Fi (Sesuaikan dengan SSID & Password Hotspot Anda)
const char* ssid     = "Kosan Vama";
const char* password = "Maira120413";

// 2. Konfigurasi Broker MQTT Lokal
// Ganti dengan Alamat IPv4 Laptop Anda yang diperoleh melalui perintah 'ipconfig'!
const char* mqttServer = "192.168.100.150";
const int   mqttPort   = 1883;

// Kredensial Autentikasi Broker MQTT (Dikonfigurasi pada Modul 7)
const char* mqttUser   = "admin";
const char* mqttPass   = "rahasia123";

// 3. Konfigurasi Identitas Node & Topik MQTT
const char* clientId      = "ESP8266_Local_Node01";
const char* topicPubTele  = "kampus/labiot/telemetri";
const char* topicSubCmd   = "kampus/labiot/kendali";

// 4. Konfigurasi Alokasi Pin Perangkat Keras
const byte dhtPin = 2; // Pin D4 (GPIO 2)
```

kode8.5.4.ino

```
// 4. Konfigurasi Alokasi Pin Perangkat Keras
const byte dhtPin = 2; // Pin D4 (GPIO 2)
const byte ledPin = 5; // Pin D1 (GPIO 5)

#define DHTTYPE DHT22  // Ubah ke DHT22 jika fisik sensor berwarna putih

DHT dht(dhtPin, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);

// Variabel Pewaktu Asinkron (Non-Blocking Timer)
unsigned long lastTeleTime = 0;
const unsigned long teleInterval = 4000; // Siklus kirim data telemetri setiap 4 detik

// Penanganan Paket Masuk (Callback Kendali Aktuator)
void onMessageCallback(char* topic, byte* payload, unsigned int length) {
  char incomingBuffer[length + 1];
  memcpy(incomingBuffer, payload, length);
  incomingBuffer[length] = '\0';

  Serial.println("\n---------------------------------------");
  Serial.printf("[KENDALI MASUK] Topik: %s\n", topic);
  Serial.printf("[PAYLOAD]       : %s\n", incomingBuffer);

  // Parsing JSON Instruksi Kendali: {"led": 1} atau {"led": 0}
  StaticJsonDocument<256> doc;
  DeserializationError err = deserializeJson(doc, incomingBuffer);
```

kode8.5.4.ino

```
  }
  Serial.println("---------------------------------------");
}

void connectToWiFi() {
  Serial.println();
  Serial.print("[Wi-Fi] Menghubungkan ke ");
  Serial.println(ssid);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\n[Wi-Fi] Berhasil Terhubung!");
  Serial.print("[Wi-Fi] IP NodeMCU: ");
  Serial.println(WiFi.localIP());
}

void reconnectMQTT() {
  while (!client.connected()) {
    Serial.print("[MQTT] Menghubungkan ke Broker Lokal (");
    Serial.print(mqttServer);
    Serial.print(")...");

    // Menghubungkan dengan kredensial resmi
    if (client.connect(clientId, mqttUser, mqttPass)) {
```

kode8.5.4.ino

```
      Serial.printf(" Gagal (rc=%d). Mencoba lagi dalam 4 detik...\n", client.state());
      delay(4000);
    }
  }
}

void setup() {
  Serial.begin(115200);

  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW); // Inisialisasi awal: LED padam

  dht.begin();
  connectToWiFi();

  client.setServer(mqttServer, mqttPort);
  client.setCallback(onMessageCallback);
  client.setBufferSize(512);
}

void loop() {
  if (WiFi.status() != WL_CONNECTED) {
    connectToWiFi();
  }

  if (!client.connected()) {
    reconnectMQTT();
  }
```

kode8.5.4.ino

```
  lastTeleTime = now;

  float t = dht.readTemperature();
  float h = dht.readHumidity();

  if (!isnan(t) && !isnan(h)) {
    // Menyusun dokumen JSON telemetri
    StaticJsonDocument<256> docOut;
    docOut["node_id"]    = clientId;
    docOut["suhu"]       = serialized(String(t, 2));
    docOut["kelembapan"] = serialized(String(h, 2));
    docOut["status_led"] = digitalRead(ledPin);
    docOut["zona"]       = "Zona-A"; // Metadata penandaan zona ruang sensor

    char teleBuffer[256];
    serializeJson(docOut, teleBuffer);

    bool success = client.publish(topicPubTele, teleBuffer);
    if (success) {
      Serial.printf("[TELEMETRI TERKIRIM] %s\n", teleBuffer);
    } else {
      Serial.println("[ERROR] Gagal memublikasikan telemetri ke broker!");
    }
  } else {
    Serial.println("[ERROR] Gagal membaca data dari sensor fisik DHT!");
  }
 }
}
```

3. Menjalankan Program dan Memeriksa Serial Monitor

Setelah program berhasil diunggah ke NodeMCU, saya membuka Serial Monitor untuk melihat proses kerja perangkat. Pada hasilnya terlihat perangkat berhasil terhubung ke jaringan dan MQTT Broker, kemudian mengirimkan data telemetri berupa suhu, kelembapan, dan status LED.



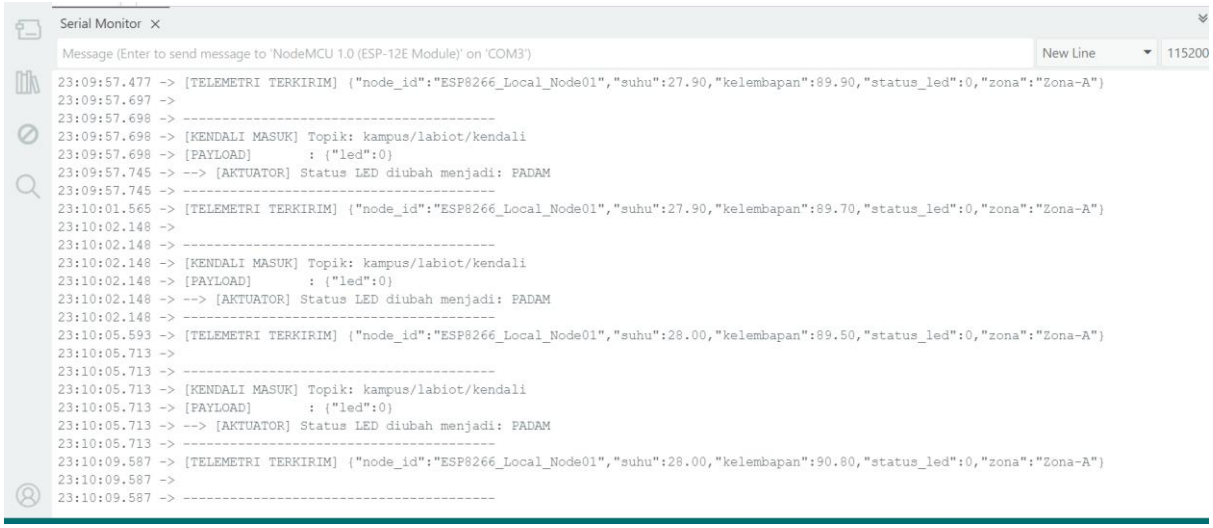
4. Menguji Pengendalian LED Berdasarkan Suhu

Pada tahap ini, saya melakukan pengujian terhadap logika pengendalian LED menggunakan batas suhu 31°C. Ketika suhu berada di atas 31°C, LED akan menyala, sedangkan ketika suhu berada di bawah batas tersebut, LED berada dalam kondisi mati.

Ketika suhu di atas 31.00



Ketika suhu di bawah 31.00

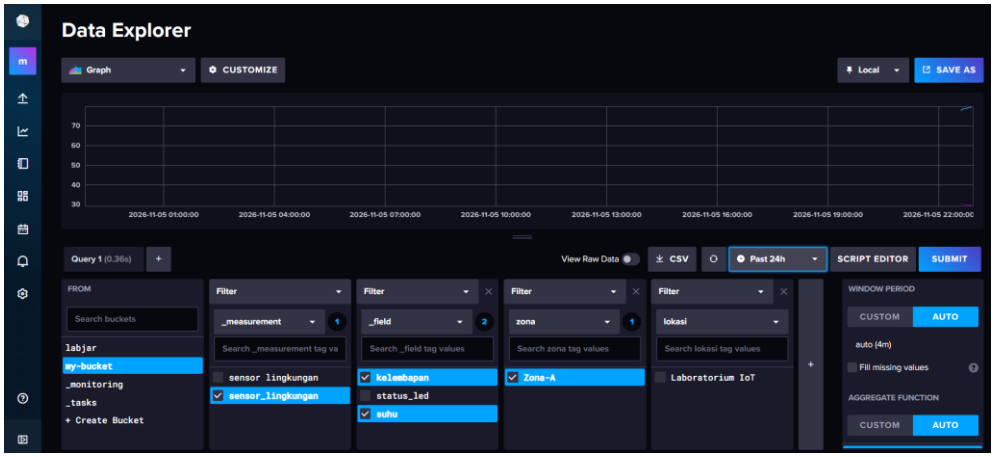


5. Memeriksa Hasil Pengiriman Data ke InfluxDB

Setelah proses pengiriman data berjalan, saya membuka InfluxDB Data Explorer untuk memastikan data sensor berhasil tersimpan. Pada tampilan ini terlihat data telemetri yang masuk, seperti nilai suhu, kelembapan, dan informasi pengukuran lainnya.

6. Menampilkan Data dalam Bentuk Grafik Garis

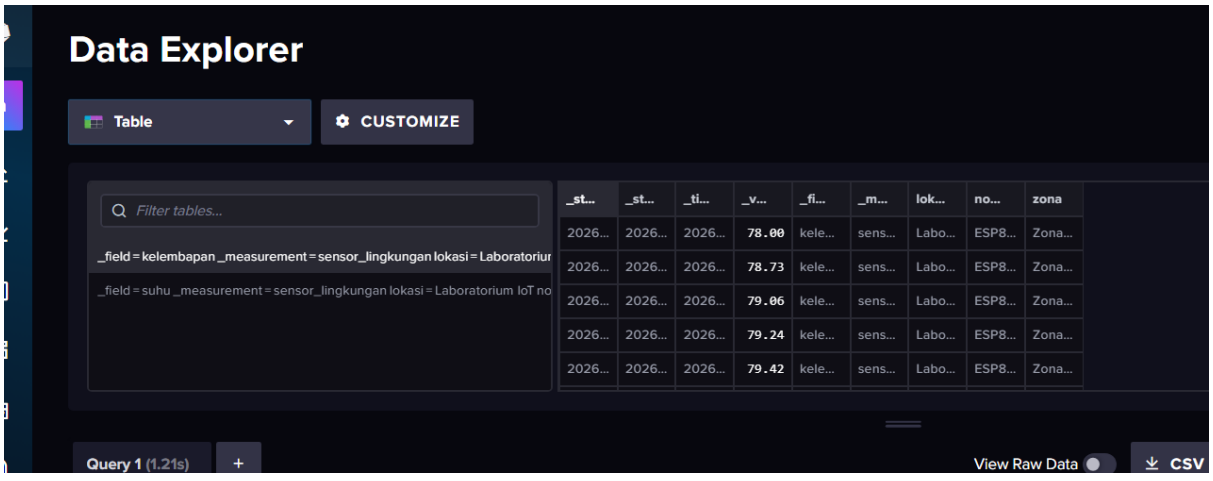
Selanjutnya, saya menampilkan data sensor menggunakan visualisasi grafik garis pada InfluxDB. Grafik tersebut memperlihatkan perubahan nilai pengukuran berdasarkan waktu sehingga kondisi sensor dapat dipantau dengan lebih mudah.

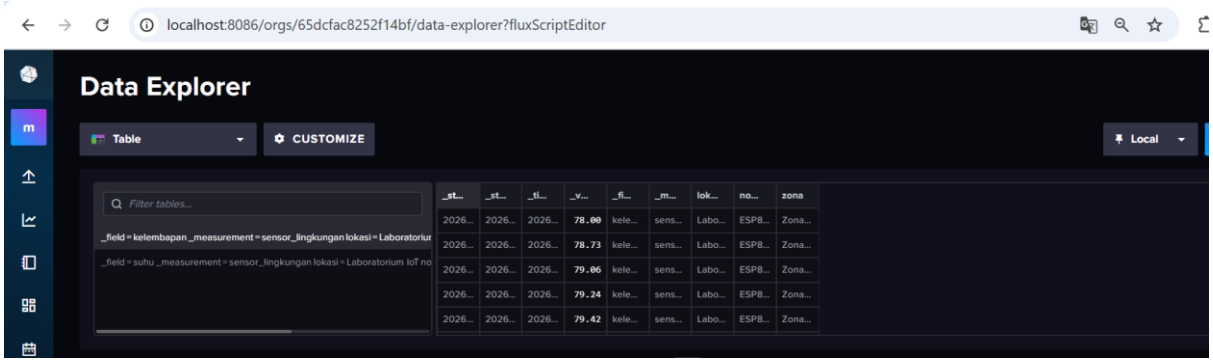




7. Menampilkan Data dalam Bentuk Tabel

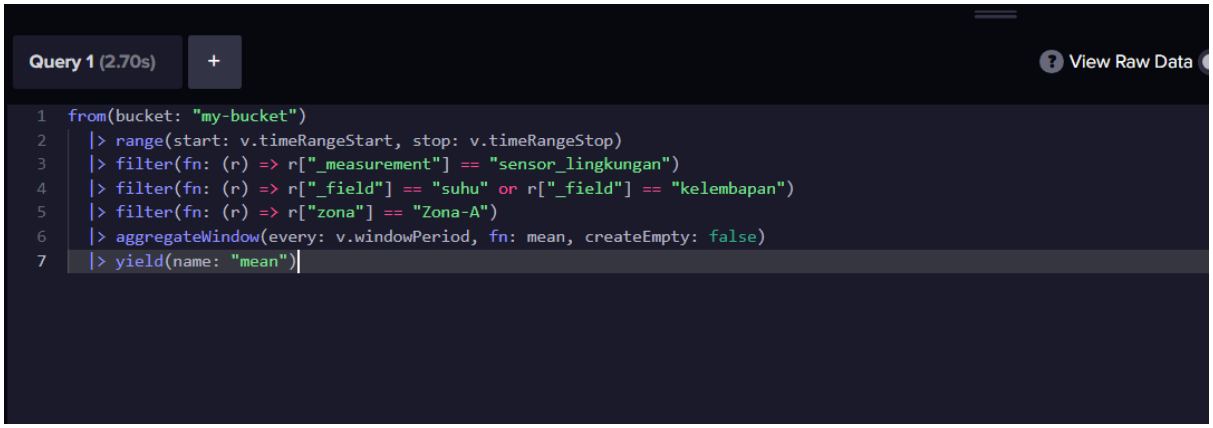
Selain grafik, saya juga menampilkan hasil pengukuran dalam bentuk tabel. Tampilan ini membantu saya melihat data secara lebih rinci, termasuk waktu pengiriman dan nilai pengukuran sensor yang telah tersimpan di database.





8. Memeriksa Kode Flux di Script Editor

Script Editor menampilkan kode Flux. Skrip ini mengambil data dari bucket my-bucket, memfilter measurement sensor lingkungan, mengambil field suhu dan kelembapan, lalu menghitung nilai rata-rata dalam interval waktu.



8.6 Latihan

Latihan 1. Analisis Kinerja dan Efisiensi Penyimpanan: TSDB vs RDBMS

Sebuah pabrik cerdas memasang 100 simpul sensor NodeMCU ESP8266. Setiap simpul mengirimkan data telemetri yang terdiri dari 3 metrik *float* (suhu, kelembapan, getaran mesin) setiap 2 detik secara kontinu selama 30 hari penuh.

1. Hitung jumlah total titik data (*data points*) yang harus disimpan oleh sistem basis data selama periode 30 hari tersebut!

   Jawab:

   Diketahui:

- Jumlah NodeMCU ESP8266: 100 node
- Jumlah metrik setiap node: 3 metrik, yaitu suhu, kelembapan, dan getaran mesin
- Interval pengiriman data: setiap 2 detik
- Periode penyimpanan: 30 hari

  Penyelesaian:

  Jumlah detik dalam 30 hari:

  30 × 24 × 60 × 60 = 2.592.000 detik

  Jumlah pengiriman data setiap node:

  2.592.000 ÷ 2 = 1.296.000 kali

  Jumlah pengiriman dari 100 node:

1.296.000 × 100 = 129.600.000 kali

Karena setiap pengiriman membawa 3 metrik, jumlah total nilai metrik yang dihasilkan adalah:

129.600.000 × 3 = 388.800.000 nilai metrik

Jawaban:

Selama 30 hari, sistem menghasilkan 129.600.000 kali pengiriman telemetri. Karena setiap pengiriman terdiri dari 3 metrik, jumlah total data points berdasarkan setiap nilai metrik adalah **388.800.000 data points**.

2. Jika menggunakan RDBMS konvensional dengan indeks *B-Tree*, rata-rata ukuran per baris transaksi adalah sekitar 150 byte. Sedangkan pada InfluxDB v2 dengan kompresi *Gorilla* dan *Delta*, ukuran per titik data terkompresi hanya sekitar 2,5 byte. Berapa rasio penghematan ruang penyimpanan (*storage savings*) yang dihasilkan oleh InfluxDB dibandingkan RDBMS?

   Jawab:

   Diketahui:

- Ukuran rata-rata setiap baris pada RDBMS: 150 byte
- Ukuran setiap data point terkompresi pada InfluxDB: 2,5 byte
- Jumlah data points: 388.800.000

  Ukuran penyimpanan pada RDBMS:

  388.800.000 × 150 = 58.320.000.000 byte

  Ukuran penyimpanan pada InfluxDB:

  388.800.000 × 2,5 = 972.000.000 byte

  Rasio penghematan penyimpanan:

  150 ÷ 2,5 = 60 kali

  Persentase penghematan:

  (150 − 2,5) ÷ 150 × 100% = 98,33%

  Jawaban:

  Berdasarkan asumsi ukuran data pada soal, InfluxDB membutuhkan ruang penyimpanan sekitar 60 kali lebih kecil dibandingkan RDBMS. Persentase penghematan ruang penyimpanan yang dihasilkan adalah sekitar **98,33%**.

3. Jelaskan mengapa struktur indeks *B-Tree* pada RDBMS mengalami penurunan performa drastis (*write degradation*) saat menangani laju data konstan tersebut!

Jawab:

Indeks B-Tree pada RDBMS digunakan untuk mempercepat proses pencarian data. Namun, ketika sistem menerima data telemetri dalam jumlah besar secara terus-menerus, indeks tersebut juga harus diperbarui setiap kali data baru dimasukkan.

Proses pembaruan indeks membutuhkan operasi tambahan, seperti mencari posisi data, memperbarui struktur indeks, dan melakukan penulisan ke disk. Pada kondisi penulisan yang sangat tinggi, proses tersebut dapat meningkatkan beban I/O, menyebabkan persaingan akses terhadap sumber daya, dan memperlambat proses penyimpanan data.

InfluxDB merupakan basis data yang dirancang khusus untuk menangani data time-series. Struktur penyimpanannya lebih sesuai untuk data yang terus bertambah berdasarkan waktu, sehingga dapat menangani penulisan telemetri secara lebih efisien.

Dengan demikian, penurunan performa pada RDBMS bukan berarti B-Tree tidak mampu menangani data besar, tetapi beban pemeliharaan indeks dapat menjadi lebih tinggi ketika sistem menerima data secara terus-menerus dalam jumlah besar.

Latihan 2. Fenomena Cardinality Explosion pada InfluxDB Tags

Seorang pengembang IoT pemula memasukkan nilai acak presisi tinggi seperti nomor transaksi UUID (uuid=a8f9-c412...) atau nilai pembacaan sensor pecahan (suhu_str=29.84512) ke dalam **Tag Set** InfluxDB, bukan ke dalam *Field Set*.

1. Jelaskan bagaimana mekanisme *Inverted Index* pada InfluxDB bekerja terhadap *Tags* dan mengapa tindakan pengembang tersebut memicu fenomena **Cardinality Explosion**!

   Jawab:

   InfluxDB menggunakan Inverted Index untuk mempercepat proses pencarian data berdasarkan nilai Tag. Inverted Index memungkinkan sistem menghubungkan nilai Tag tertentu dengan data atau series yang memiliki nilai tersebut.

   Contohnya, Tag lokasi=Laboratorium IoT dapat digunakan untuk mencari data sensor dari lokasi tersebut dengan lebih cepat.

   Namun, apabila pengembang memasukkan nilai yang selalu berubah atau memiliki banyak kemungkinan nilai unik, seperti UUID transaksi atau pembacaan suhu dengan presisi tinggi, jumlah kombinasi Tag Set akan meningkat secara drastis.

Setiap kombinasi Tag yang berbeda dapat membentuk Series Key baru. Akibatnya, jumlah series yang harus dikelola oleh InfluxDB menjadi sangat besar. Fenomena inilah yang disebut **Cardinality Explosion**.

2. Apa dampak fatal fenomena **Cardinality Explosion** terhadap konsumsi memori utama (RAM) peladen dan stabilitas kontainer Docker InfluxDB?

   Jawab:

   Cardinality Explosion dapat menyebabkan peningkatan penggunaan RAM karena InfluxDB harus menyimpan dan mengelola indeks untuk jumlah series yang semakin besar.

   Dampak yang dapat terjadi antara lain:

1. Konsumsi RAM meningkat secara signifikan.
2. Proses pencarian dan penulisan data menjadi lebih berat.
3. Kinerja InfluxDB mengalami penurunan.
4. Kontainer Docker berisiko mengalami kehabisan memori atau Out of Memory (OOM).
5. Jika penggunaan memori melebihi batas yang tersedia, kontainer dapat berhenti atau mengalami restart.

   Oleh karena itu, pengelolaan Tag perlu dilakukan dengan baik agar jumlah series tidak berkembang secara berlebihan.

3. Rumuskan pedoman baku bagi seorang insinyur sistem untuk menentukan kapan sebuah variabel data wajib dijadikan *Tag* dan kapan wajib dijadikan *Field*!

4. Variabel	Penyimpanan	Alasan
Lokasi sensor	Tag	Nilainya relatif tetap dan sering digunakan untuk filter
ID perangkat	Tag	Jumlah perangkat biasanya terbatas dan berguna untuk pengelompokan
Zona sensor	Tag	Digunakan untuk mengelompokkan data berdasarkan zona
Suhu	Field	Merupakan nilai pengukuran yang berubah-ubah
Kelembapan	Field	Merupakan nilai pengukuran sensor
Getaran mesin	Field	Merupakan data hasil pengukuran
UUID transaksi	Field	Memiliki banyak nilai unik dan berpotensi meningkatkan cardinality jika dijadikan Tag
Timestamp	Timestamp data	Digunakan sebagai penanda waktu pengukuran

Jawaban:

Tag sebaiknya digunakan untuk metadata yang memiliki jumlah nilai unik relatif terbatas dan sering digunakan untuk pencarian atau pengelompokan data. Sementara itu, Field digunakan untuk menyimpan nilai pengukuran yang berubah-ubah atau memiliki banyak nilai unik, seperti suhu, kelembapan, dan getaran mesin.

Latihan 3. Arsitektur Jaringan Docker dan Resolusi Nama Layanan DNS

Pada Praktikum 3, di dalam konfigurasi node influxdb out pada antarmuka Node-RED, kita mengisi alamat server dengan URL **http://influxdb:8086**, sedangkan pada peramban web laptop kita mengakses InfluxDB melalui **http://localhost:8086**.

1. Mengapa penulisan URL http://localhost:8086 di dalam konfigurasi node influxdb out pada Node-RED akan menyebabkan kegagalan koneksi (*Connection Refused / ETIMEDOUT*)?

   Jawab:

   Pada lingkungan Docker, setiap kontainer memiliki lingkungan jaringan masing-masing. Oleh karena itu, alamat localhost mengacu pada kontainer tempat aplikasi tersebut berjalan.

   Ketika Node-RED menggunakan URL http://localhost:8086, alamat tersebut mengarah ke kontainer Node-RED sendiri, bukan ke kontainer InfluxDB.

   Karena layanan InfluxDB berjalan pada kontainer yang berbeda, Node-RED tidak dapat menemukan layanan tersebut melalui alamat localhost. Akibatnya, koneksi dapat gagal dengan pesan Connection Refused atau ETIMEDOUT.

   Alamat yang benar untuk komunikasi antar-kontainer adalah:

   http://influxdb:8086

   Alamat tersebut menggunakan nama layanan Docker influxdb, sehingga Node-RED dapat mengakses layanan InfluxDB melalui jaringan internal Docker.

2. Jelaskan peran *Docker Embedded DNS Resolver* (127.0.0.11) dan jaringan *bridge* (iot-net) dalam memfasilitasi komunikasi antar-kontainer tanpa harus mengekspos port ke jaringan publik!

   Jawab:

   Docker menyediakan Embedded DNS Resolver dengan alamat 127.0.0.11 pada jaringan internal kontainer. DNS ini berfungsi untuk menerjemahkan nama layanan Docker menjadi alamat IP kontainer yang sesuai.

   Sebagai contoh, ketika Node-RED mengakses http://influxdb:8086, Docker akan menerjemahkan nama layanan influxdb menjadi alamat IP kontainer InfluxDB.

Jaringan bridge seperti iot-net memungkinkan kontainer yang tergabung di dalam jaringan tersebut saling berkomunikasi menggunakan nama layanan Docker.

Dengan adanya jaringan ini, Node-RED dan InfluxDB dapat bertukar data tanpa harus menggunakan alamat IP secara manual. Komunikasi internal antar-kontainer juga tidak harus melewati port yang dipublikasikan ke jaringan luar.

Docker Embedded DNS Resolver membantu menerjemahkan nama layanan menjadi alamat IP kontainer, sedangkan jaringan bridge iot-net memungkinkan komunikasi antar-kontainer secara internal. Oleh karena itu, Node-RED dapat mengakses InfluxDB menggunakan nama layanan influxdb tanpa perlu menggunakan localhost.

Latihan 4. Strategi Retensi Data dan Agregasi Berjenjang (*Downsampling*)

Dalam arsitektur IoT industri, data telemetri berfrekuensi tinggi (sampel per detik) sangat penting untuk investigasi cepat jika terjadi gangguan mesin dalam 7 hari terakhir. Namun, untuk pelaporan kinerja tahunan (*annual KPI*), manajemen hanya membutuhkan nilai rata-rata per jam.

1. Mengapa menyimpan data berpresisi detik selama bertahun-tahun dianggap sebagai pemborosan sumber daya komputasi (*inefficient architecture*)?

   Jawab:

   Data telemetri yang dikirim setiap detik akan menghasilkan jumlah data yang sangat besar. Jika seluruh data mentah tersebut disimpan selama bertahun-tahun, kebutuhan ruang penyimpanan dan sumber daya komputasi akan terus meningkat.

   Padahal, untuk kebutuhan laporan tahunan atau perhitungan KPI, manajemen biasanya hanya membutuhkan data ringkasan, seperti rata-rata suhu setiap jam, bukan seluruh data pengukuran per detik.

   Oleh karena itu, menyimpan seluruh data mentah dalam jangka panjang dapat menjadi pemborosan sumber daya. Data mentah tetap penting untuk investigasi gangguan dalam waktu dekat, sedangkan data lama dapat diringkas sesuai kebutuhan analisis.

2. Jelaskan konsep **Retention Policy** pada *Bucket* InfluxDB v2!

   Jawab:

Retention Policy merupakan aturan yang menentukan berapa lama data disimpan dalam sebuah Bucket InfluxDB.

Pada InfluxDB v2, aturan retensi diterapkan melalui konfigurasi Bucket. Data yang telah melewati periode retensi dapat dihapus secara otomatis oleh InfluxDB.

Contohnya:

- bucket_raw_7d digunakan untuk menyimpan data telemetri mentah selama 7 hari.
- bucket_aggregated_1y digunakan untuk menyimpan data hasil agregasi selama 1 tahun.

Dengan strategi tersebut, data mentah yang sudah tidak diperlukan dapat dihapus sesuai masa retensinya, sedangkan data hasil agregasi tetap tersedia untuk kebutuhan pelaporan jangka panjang.

3. Rancang strategi arsitektur dua *bucket* (*Dual-Bucket Strategy*): jelaskan bagaimana bahasa skrip Flux dapat diprogram sebagai tugas berkala (*InfluxDB Task*) untuk melakukan *downsampling* otomatis dari bucket_raw_7d ke bucket_aggregated_1y!

   Jawab:

   Strategi Dual-Bucket menggunakan dua Bucket dengan fungsi yang berbeda, yaitu Bucket untuk data mentah dan Bucket untuk data hasil agregasi.

   Bucket pertama: bucket_raw_7d

   Bucket ini digunakan untuk menyimpan data telemetri mentah yang memiliki frekuensi pengiriman tinggi.

   Data pada Bucket ini digunakan untuk:

   - Memantau kondisi sensor secara detail.
   - Menganalisis gangguan mesin.
   - Melakukan investigasi terhadap kejadian dalam 7 hari terakhir.

   Bucket kedua: bucket_aggregated_1y

   Bucket ini digunakan untuk menyimpan data hasil agregasi, seperti rata-rata suhu, kelembapan, dan getaran mesin setiap jam.

   Data pada Bucket ini digunakan untuk:

   - Membuat laporan tahunan.
   - Menganalisis tren kinerja mesin.

- Menghitung KPI.
- Melakukan evaluasi kondisi mesin dalam jangka panjang.

Alur proses downsampling:

1. NodeMCU mengirimkan data telemetri ke sistem.
2. Data mentah disimpan ke dalam Bucket bucket_raw_7d.
3. InfluxDB menjalankan Task Flux secara berkala.
4. Task membaca data mentah dari Bucket pertama.
5. Flux mengelompokkan data berdasarkan interval waktu satu jam.
6. Flux menghitung nilai rata-rata setiap metrik.
7. Hasil agregasi ditulis ke Bucket bucket_aggregated_1y.
8. Data mentah yang sudah melewati masa retensi 7 hari akan dihapus secara otomatis.

Contoh skrip Flux untuk melakukan agregasi rata-rata suhu setiap jam:

```
option task = {
    name: "Downsampling Telemetri Per Jam",
    every: 1h
}


from(bucket: "bucket_raw_7d")
    |> range(start: -task.every)
    |> filter(fn: (r) =>
        r._measurement == "sensor_lingkungan"
    )
    |> filter(fn: (r) =>
        r._field == "suhu" or
        r._field == "kelembapan" or
        r._field == "getaran"
    )
    |> aggregateWindow(
```

```
    every: 1h,
    fn: mean,
    createEmpty: false
  )
  |> to(
    bucket: "bucket_aggregated_1y",
    org: "my-org"
  )
```

Skrip tersebut mengambil data telemetri dari Bucket mentah, kemudian mengelompokkan data berdasarkan interval satu jam dan menghitung nilai rata-ratanya. Hasil agregasi selanjutnya disimpan ke Bucket kedua.

Skrip tersebut merupakan contoh rancangan Task Flux. Agar dapat dijalankan, kedua Bucket harus sudah dibuat dan konfigurasi Task perlu disesuaikan dengan organisasi serta struktur data yang digunakan.

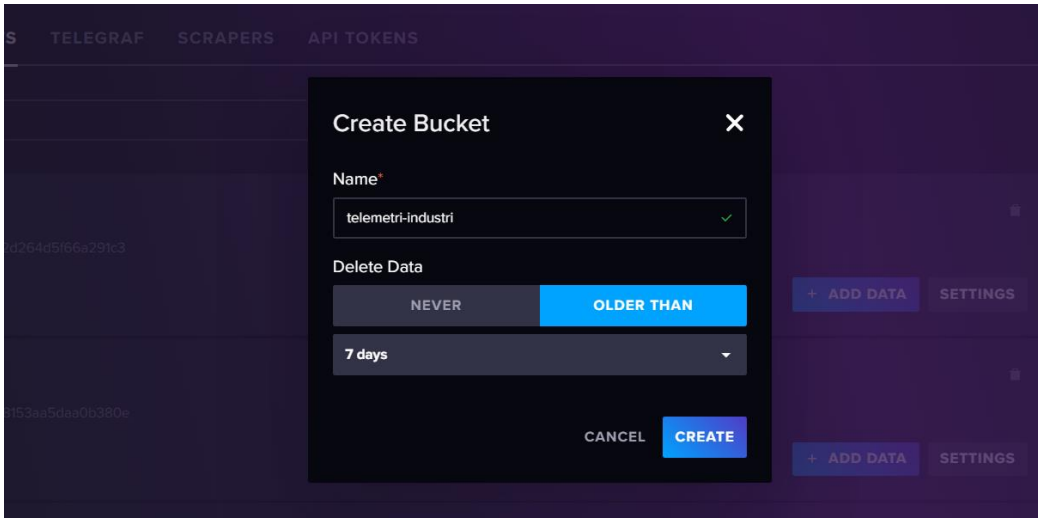
Jawaban:

Strategi Dual-Bucket memungkinkan sistem tetap menyimpan data mentah untuk kebutuhan investigasi jangka pendek dan menyimpan data agregasi untuk analisis jangka panjang. Dengan bantuan Task Flux, proses downsampling dapat dilakukan secara otomatis sehingga penggunaan ruang penyimpanan menjadi lebih efisien.

8.7 Tugas: Implementasi Metadata Tagging Multi-Zona dan Pemfilteran Kueri Telemetri

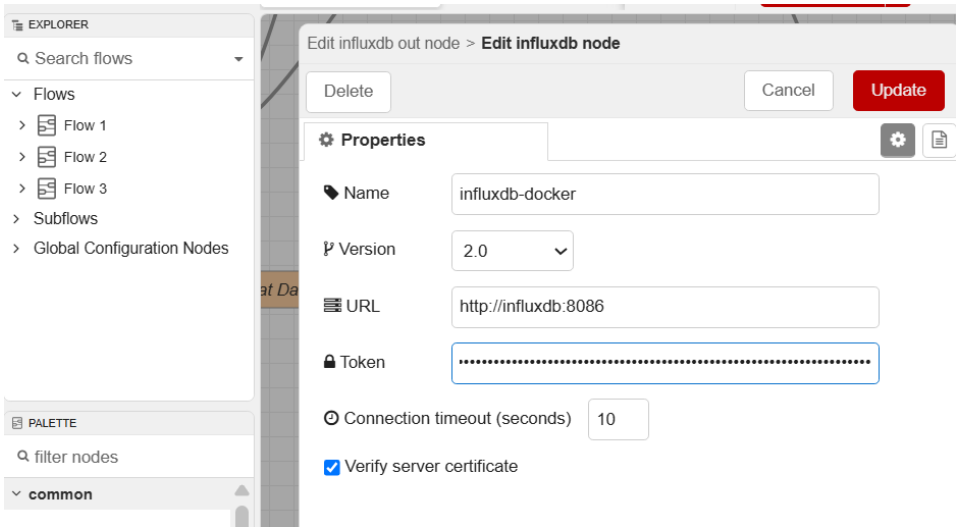
1. Membuat Bucket Khusus Telemetri Industri

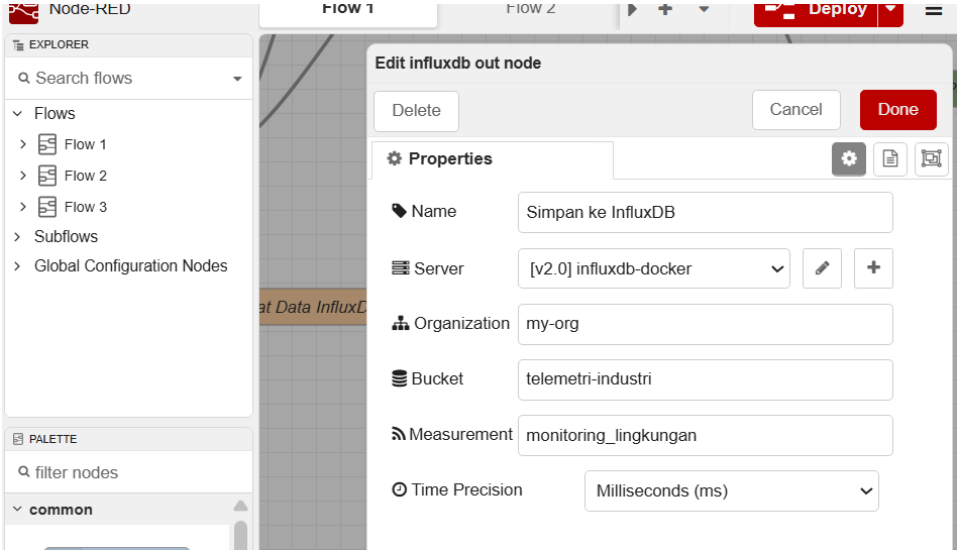
Saya membuat bucket baru dengan nama telemetri-industri pada InfluxDB sebagai tempat penyimpanan data telemetri industri. Saya juga mengatur kebijakan retensi data menjadi 7 hari agar data yang sudah melewati batas waktu penyimpanan dapat dihapus secara otomatis.



2. Mengatur Koneksi InfluxDB pada Node-RED

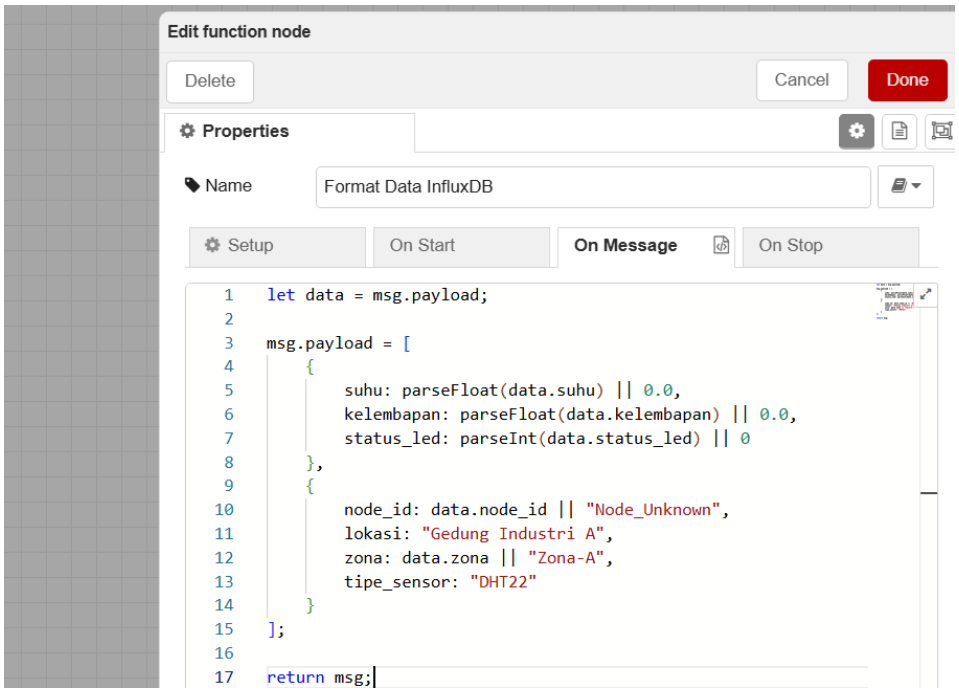
Selanjutnya, saya mengatur kembali konfigurasi InfluxDB Out Node pada Node-RED agar terhubung ke bucket telemetri-industri. Saya menggunakan Organization my-org dan Scoped API Token yang telah dibuat agar proses pengiriman data dapat dilakukan dengan hak akses yang sesuai.





3. Membuat Function untuk Metadata Tagging

Pada tahap ini, saya menyesuaikan Function Node Format Data InfluxDB untuk memisahkan data pengukuran dan metadata. Nilai suhu, kelembapan, serta status LED dimasukkan ke dalam Field, sedangkan informasi node_id, lokasi, zona, dan tipe_sensor digunakan sebagai Tag.



4. Menambahkan Metadata pada Program ESP8266

Selanjutnya, saya menyesuaikan program NodeMCU agar data JSON yang dikirimkan melalui MQTT memiliki informasi tambahan berupa identitas perangkat dan zona sensor. Dengan penambahan metadata tersebut, data yang diterima InfluxDB dapat dikelompokkan berdasarkan perangkat maupun lokasi pengukuran.

NodeMCU 1.0 (ESP-12...

kode8.5.4.ino

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>

// 1. Kredensial Jaringan Wi-Fi (Sesuaikan dengan SSID & Password Hotspot Anda)
const char* ssid     = "MAMA NAELA";
const char* password = "Naela090106(";

// 2. Konfigurasi Broker MQTT Lokal
// Ganti dengan Alamat IPv4 Laptop Anda yang diperoleh melalui perintah 'ipconfig'!
const char* mqttServer = "192.168.1.7";
const int   mqttPort   = 1883;

// Kredensial Autentikasi Broker MQTT (Dikonfigurasi pada Modul 7)
const char* mqttUser   = "admin";
const char* mqttPass   = "rahasia123";

// 3. Konfigurasi Identitas Node & Topik MQTT
const char* clientId      = "ESP8266_Local_Node01";
const char* topicPubTele  = "kampus/labiot/telemetri";
const char* topicSubCmd   = "kampus/labiot/kendali";

// 4. Konfigurasi Alokasi Pin Perangkat Keras
const byte dhtPin = 2; // Pin D4 (GPIO 2)
```

kode8.5.4.ino

```
const byte dhtPin = 2; // Pin D4 (GPIO 2)
const byte ledPin = 5; // Pin D1 (GPIO 5)

#define DHTTYPE DHT22  // Ubah ke DHT22 jika fisik sensor berwarna putih

DHT dht(dhtPin, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);

// Variabel Pewaktu Asinkron (Non-Blocking Timer)
unsigned long lastTeleTime = 0;
const unsigned long teleInterval = 4000; // Siklus kirim data telemetri setiap 4 detik

// Penanganan Paket Masuk (Callback Kendali Aktuator)
void onMessageCallback(char* topic, byte* payload, unsigned int length) {
  char incomingBuffer[length + 1];
  memcpy(incomingBuffer, payload, length);
  incomingBuffer[length] = '\0';

  Serial.println("\n----------------------------------------");
  Serial.printf("[KENDALI MASUK] Topik: %s\n", topic);
  Serial.printf("[PAYLOAD]       : %s\n", incomingBuffer);

  // Parsing JSON Instruksi Kendali: {"led": 1} atau {"led": 0}
  StaticJsonDocument<256> doc;
```

kode8.5.4.ino

```
134
135  // Pengiriman Telemetri Periodik Non-Blocking
136  unsigned long now = millis();
137  if (now - lastTeleTime >= teleInterval) {
138    lastTeleTime = now;
139
140    float t = dht.readTemperature();
141    float h = dht.readHumidity();
142
143    if (!isnan(t) && !isnan(h)) {
144      // Menyusun dokumen JSON telemetri
145      StaticJsonDocument<256> docOut;
146      docOut["node_id"]    = clientId;
147      docOut["suhu"]       = serialized(String(t, 2));
148      docOut["kelembapan"] = serialized(String(h, 2));
149      docOut["status_led"] = digitalRead(ledPin);
150      docOut["zona"]       = "Zona-A"; // Metadata penandaan zona ruang sensor
151
152      char teleBuffer[256];
153      serializeJson(docOut, teleBuffer);
154
155      bool success = client.publish(topicPubTele, teleBuffer);
156      if (success) {
157        Serial.printf("[TELEMETRI TERKIRIM] %s\n", teleBuffer);
158      } else {
```

5. Menjalankan Program dan Memeriksa Hasil Telemetri

Setelah program dijalankan, saya memeriksa Serial Monitor untuk memastikan perangkat berhasil mengirimkan data telemetri secara berkala. Pada hasilnya terlihat informasi node_id, suhu, kelembapan, status LED, dan zona sensor berhasil disertakan dalam payload JSON. Disini saya menggantti wifi berbeda dengan praktikum 4.

Serial Monitor × Output

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3') — New Line

```
19:47:50.510 -> .....rtUtUrU$UUn♦t♦b|♦♦LLLL♦rUb♦bLLnnUlnnZbbUpU$bUtripUnUL♦t♦Ubn♦n♦
19:48:17.893 -> [Wi-Fi] Menghubungkan ke MAMA NAELA
19:48:18.489 -> ..........................................................
19:49:18.864 -> [Wi-Fi] Berhasil Terhubung!
19:49:18.864 -> [Wi-Fi] IP NodeMCU: 192.168.1.8
19:49:18.864 -> [MQTT] Menghubungkan ke Broker Lokal (192.168.1.7)... Sukses Terhubung!
19:49:21.538 -> [MQTT] Berlangganan topik kendali: kampus/labiot/kendali
19:49:21.538 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":30.30,"kelembapan":76.10,"status_led":0,"zona":"Zona-A"}
19:49:21.770 ->
19:49:21.770 -> ----------------------------------------
19:49:21.770 -> [KENDALI MASUK] Topik: kampus/labiot/kendali
19:49:21.770 -> [PAYLOAD]       : {"led":0}
19:49:21.770 -> --> [AKTUATOR] Status LED diubah menjadi: PADAM
19:49:21.810 -> ----------------------------------------
19:49:25.576 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":30.30,"kelembapan":76.20,"status_led":0,"zona":"Zona-A"}
19:49:25.609 ->
19:49:25.609 -> ----------------------------------------
19:49:25.648 -> [KENDALI MASUK] Topik: kampus/labiot/kendali
19:49:25.648 -> [PAYLOAD]       : {"led":0}
19:49:25.648 -> --> [AKTUATOR] Status LED diubah menjadi: PADAM
19:49:25.648 -> ----------------------------------------
19:49:29.564 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":30.20,"kelembapan":76.20,"status_led":0,"zona":"Zona-A"}
19:49:29.618 ->
19:49:29.618 -> ----------------------------------------
```

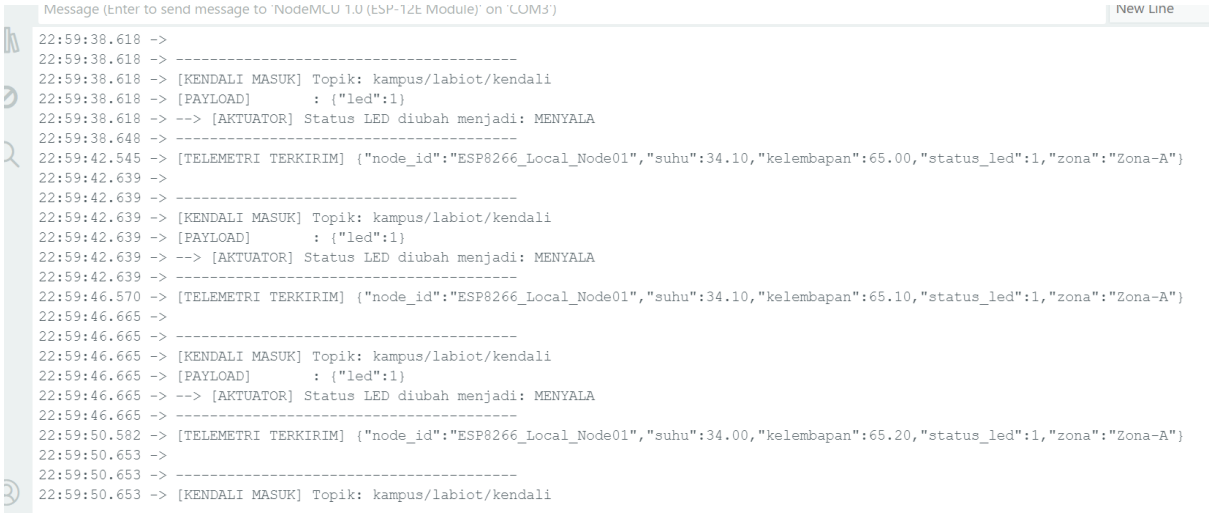
6. Menguji Pengendalian LED

Pada tahap ini, saya melakukan pengujian terhadap perubahan status LED berdasarkan nilai suhu yang terbaca oleh sensor. Ketika suhu melewati batas 31°C, LED berubah menjadi menyala, dan begitupun sebaliknya sehingga proses pembacaan sensor dan pengendalian aktuator dapat diamati melalui Serial Monitor.

padam

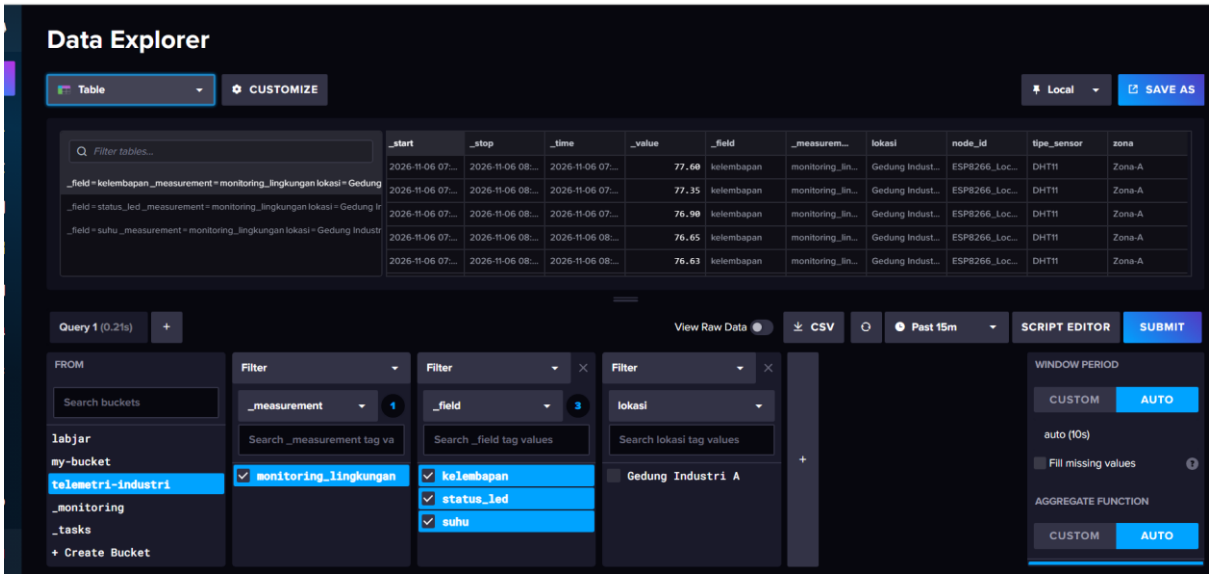


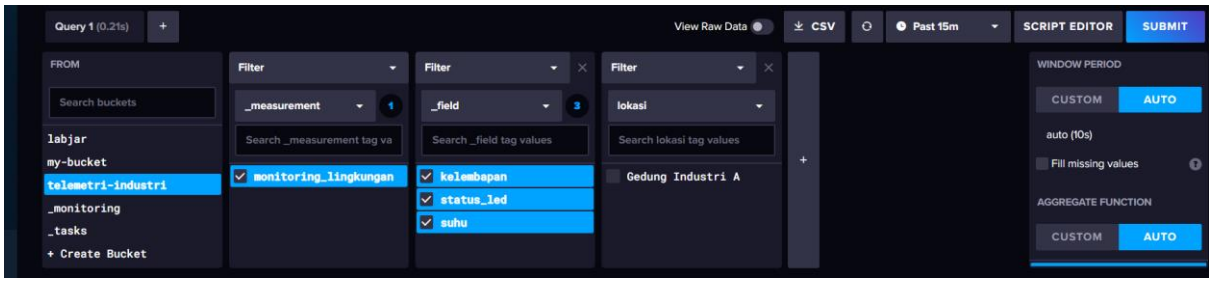
Menyala

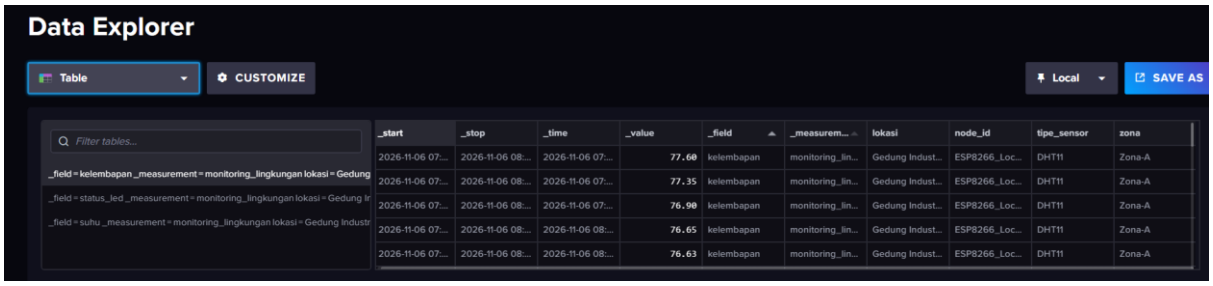


7. Memastikan Data Berhasil Tersimpan di InfluxDB

Selanjutnya, saya membuka Data Explorer pada InfluxDB untuk memastikan data dari NodeMCU berhasil masuk ke bucket telemetri-industri. Pada tampilan tabel terlihat data pengukuran beserta metadata seperti zona, lokasi, dan identitas perangkat telah tersimpan.

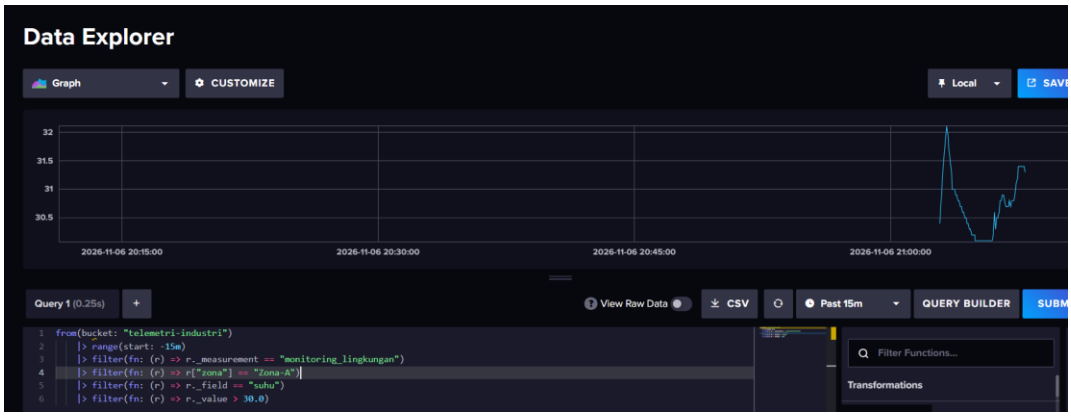


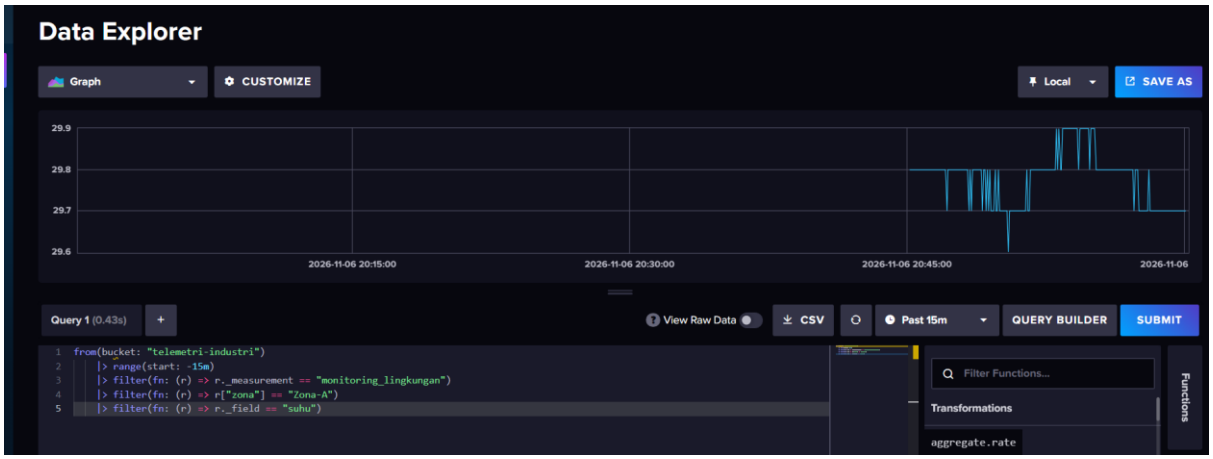




8. Menjalankan Kueri Flux untuk Zona-A

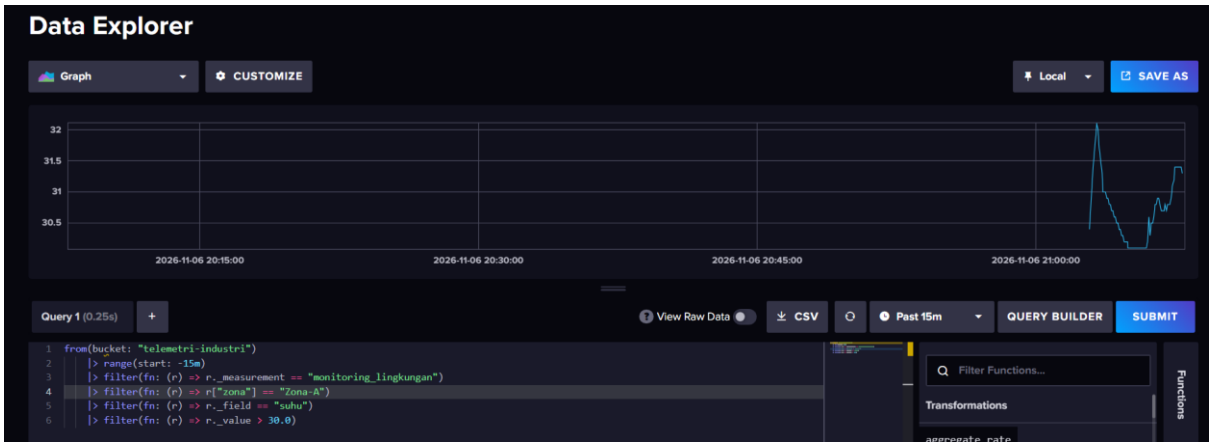
saya menggunakan Script Editor untuk menjalankan kueri Flux yang berfungsi memfilter data berdasarkan metadata Zona-A. Kueri tersebut digunakan agar sistem hanya menampilkan data telemetri yang berasal dari zona yang dipilih.





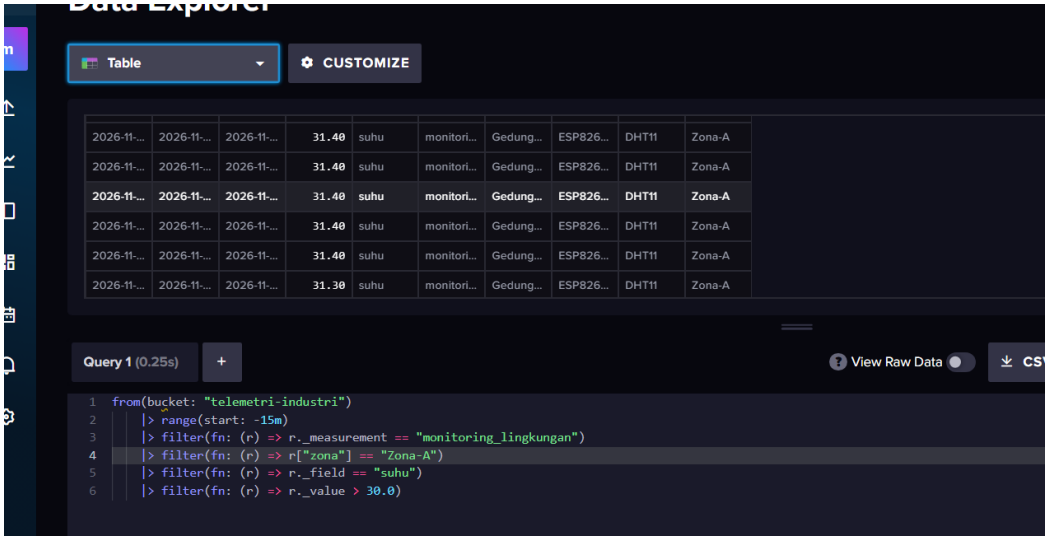
9. Memfilter Data Suhu di Atas 30°C

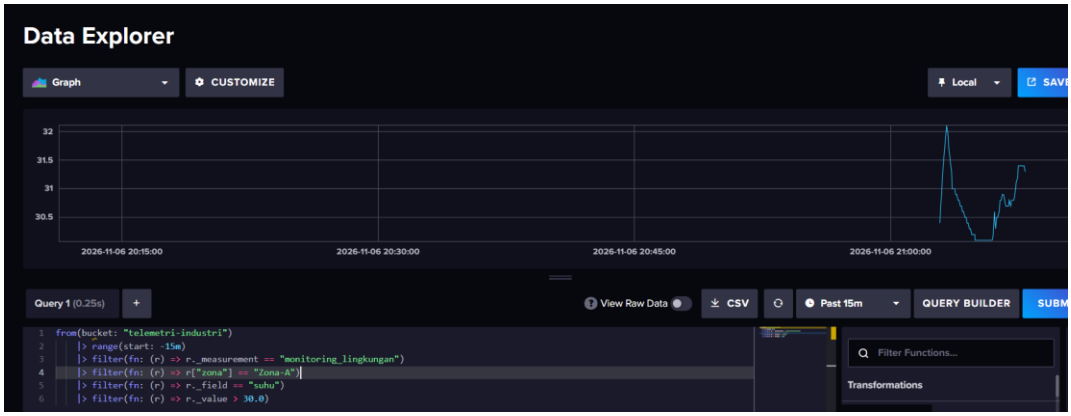
Selanjutnya, saya menambahkan kondisi pemfilteran untuk menampilkan data suhu yang memiliki nilai lebih dari 30°C. Hasilnya ditampilkan dalam bentuk grafik sehingga perubahan suhu yang melewati batas tersebut dapat diamati berdasarkan waktu pengukuran.



10. Menampilkan Hasil Kueri dalam Bentuk Tabel

Terakhir, saya memeriksa hasil kueri melalui tampilan tabel untuk melihat data yang memenuhi kondisi pemfilteran. Dengan hasil ini, saya dapat memastikan bahwa proses penyimpanan, pemberian metadata, dan pemfilteran data telemetri berdasarkan zona telah berhasil dilakukan.





KESIMPULAN

Berdasarkan praktikum Modul 8 IoT yang telah dilakukan, dapat disimpulkan bahwa proses pengolahan dan pemantauan data telemetri berhasil dipahami melalui penggunaan Docker, MQTT, Node-RED, InfluxDB, dan Grafana. Data sensor DHT22 berupa suhu dan kelembapan dapat diterima melalui MQTT, kemudian diolah menggunakan Node-RED dan disimpan ke dalam InfluxDB.

Selain itu, praktikum ini memberikan pemahaman mengenai penggunaan metadata untuk mengidentifikasi perangkat dan zona pengukuran, serta penerapan kueri Flux untuk memfilter data berdasarkan kondisi tertentu. Melalui seluruh rangkaian praktikum dan latihan, diperoleh pengalaman dalam membangun sistem pemantauan lingkungan berbasis IoT secara terintegrasi.